System Design

Backstage: a practice space for public speaking

Item	Detail
Project	Project 8 (Public Speaking Training App)
Teams	Frontend Group 7, Backend Group 8
Version	Final, written after the backend was built and tested
Date	20 October 2026
Companion documents	PRD, Project Log, backend README

1. Overview

Backstage lets a person rehearse a speech alone and get measurable feedback. They record themselves, the backend turns the audio into text with word timings, and then reports how fast they spoke, where they paused and which filler words (um, uh, like) they used. They can try again and watch the numbers improve.

This document describes the backend as it was actually built: who uses it, what it does, how a recording travels through it, where the data lives and how the pieces are put together. Every diagram matches the running code. Things that were planned in the PRD but not built are listed in section 9.

2. People and systems around the backend

Actor	What it is	What it can do
User	A person who practises speeches	Registers, picks a topic, uploads a recording, reads the results. Can only see their own recordings and results.
Admin	A user with staff rights	Everything a User can do, plus changing settings, global filler words, prompts and categories, and looking at all data in the admin panel.
Speech engine	Software, runs on our own machine	Whisper (through the faster-whisper library) turns audio into words with start and end times. A mock engine stands in when Whisper is not ready.
Frontend app	The web app built by Group 7	Talks to the backend only through the JSON API. A small test page inside the backend does the same, to try things out.

There are two roles only, user and admin. An admin is a normal user with staff rights, so no separate role table exists. Users never see each other's data: every recording query is filtered by the logged-in user.

3. What the system does

The use case diagram groups what people do (blue and amber), the supporting steps and conditions (purple) and the automatic processing (green). The catalogue after it explains each use case.

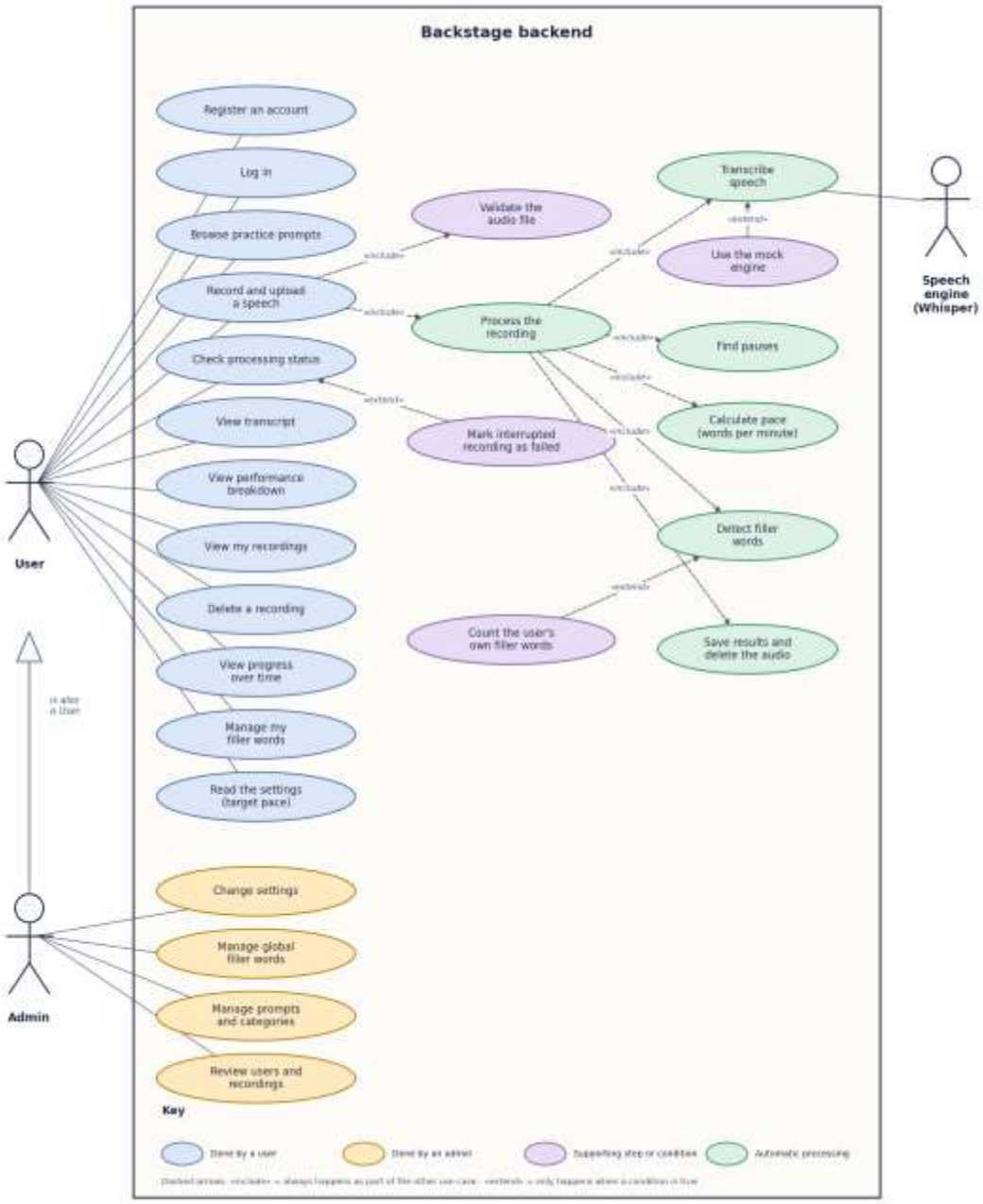


Figure 1: Use case diagram

3.1 Use case catalogue

ID	Use case	Who	What happens
UC-01	Register an account	User	Creates an account with a username and a password that passes Django's password rules.

ID	Use case	Who	What happens
UC-02	Log in	User, Admin	Exchanges username and password for an access token and a refresh token. Limited to a few attempts per minute.
UC-03	Browse practice prompts	User	Lists active prompts, with category, language and text search.
UC-04	Record and upload a speech	User	Sends an audio file. The backend answers at once and processes it in the background.
UC-05	Check processing status	User	Shows processing, completed or failed. A failed recording carries a reason.
UC-06	View transcript	User	Full text and every word with its start and end time.
UC-07	View performance breakdown	User	Pace, pace over time, pauses and filler words for one recording.
UC-08	View my recordings	User	Own recordings, newest first, optionally for one prompt.
UC-09	Delete a recording	User	Removes the recording and all its results.
UC-10	View progress over time	User	Pace, filler count and pause count of every finished attempt, for a trend chart.
UC-11	Manage my filler words	User	Adds, switches off or removes personal filler words.
UC-12	Read the settings	User	Reads thresholds such as the target pace so a screen can draw it.
UC-13	Change settings	Admin	Changes thresholds, limits and the Whisper model size.
UC-14	Manage global filler words	Admin	Edits the list used for everybody.
UC-15	Manage prompts and categories	Admin	Done in the Django admin panel.
UC-16	Review users and recordings	Admin	Done in the Django admin panel.
UC-17	Validate the audio file	System	Checks type, size and length before anything is saved.
UC-18	Process the recording	System	Background job that runs the steps below.
UC-19	Transcribe speech	Speech engine	Audio in, timed words out.
UC-20	Use the mock engine	System	Only when Whisper is not installed or its model cannot be loaded.
UC-21	Find pauses	System	Gaps between words, labelled short, dramatic or long.
UC-22	Calculate pace	System	Words per minute overall and for each time window.
UC-23	Detect filler words	System	Matches the transcript against global and personal word lists.
UC-24	Count the user's own filler words	System	Only when the user has added personal words.
UC-25	Save results and delete the audio	System	Stores transcript and numbers, removes the audio file whatever happens.
UC-26	Mark interrupted recording as failed	System	A recording still processing after 20 minutes (for example the server restarted) becomes failed.

3.2 Include, extend and generalization

"Includes" means the step always happens as part of the other use case. "Extends" means it happens only when its condition is true.

Use case	Relation	Related use case	When
Record and upload a speech	includes	Validate the audio file	Every upload is checked first.
Record and upload a speech	includes	Process the recording	Every accepted upload is processed.
Process the recording	includes	Transcribe speech, Find pauses, Calculate pace, Detect filler words, Save results and delete the audio	These five steps always run, in this order.
Use the mock engine	extends	Transcribe speech	Only if Whisper is not ready.
Count the user's own filler words	extends	Detect filler words	Only if the user added personal words.
Mark interrupted recording as failed	extends	Check processing status	Only if processing has been stuck for more than 20 minutes.
Admin	is also a	User	An admin can do everything a user can.

4. How it is built

One Django process serves the API, the admin panel and the test page. Heavy work happens in a background thread that calls the speech engine. All logic that is not about web requests sits in a plain Python package, so it can be tested on its own.

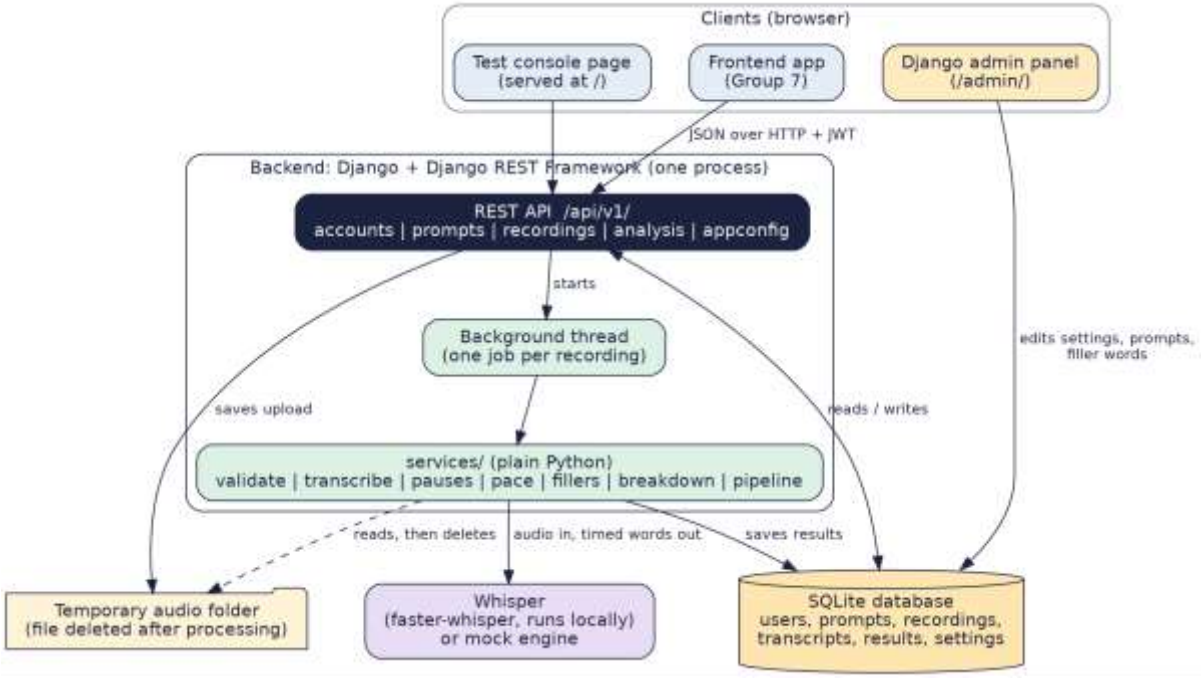


Figure 2: Architecture

Part	Role
REST API (accounts, prompts, recordings, analysis, appconfig)	Django REST Framework views. Authentication with JWT, one error format, paging, rate limits for login, register and uploads.
services/ package	Plain Python with no web code: audio length, transcription, pace, pauses, filler words, breakdown, pipeline, background jobs.
Background job	A thread per recording. Transcription is slow, so the upload returns at once and the client asks for the status. One transcription runs at a time.
Whisper (faster-whisper)	Runs locally on the CPU. Free, no account or key needed. The model is downloaded once.
SQLite database	One file. Holds users, prompts, recordings, transcripts, results and settings. Audio is never stored in it.
Temporary audio folder	The upload lives here only while it is processed, then the file is deleted.
Django admin panel	Ready-made screens for admins to edit settings, prompts and filler words.
Test console page	A small page served at / to try the whole flow in the browser. It is not the real frontend.

Technology: Python, Django 4.2 to 5.2, Django REST Framework, SimpleJWT for tokens, django-cors-headers, SQLite, faster-whisper. The project runs on Python 3.9 to 3.14 (all tested). Everything is free and runs offline once the Whisper model is downloaded.

5. How a recording is processed

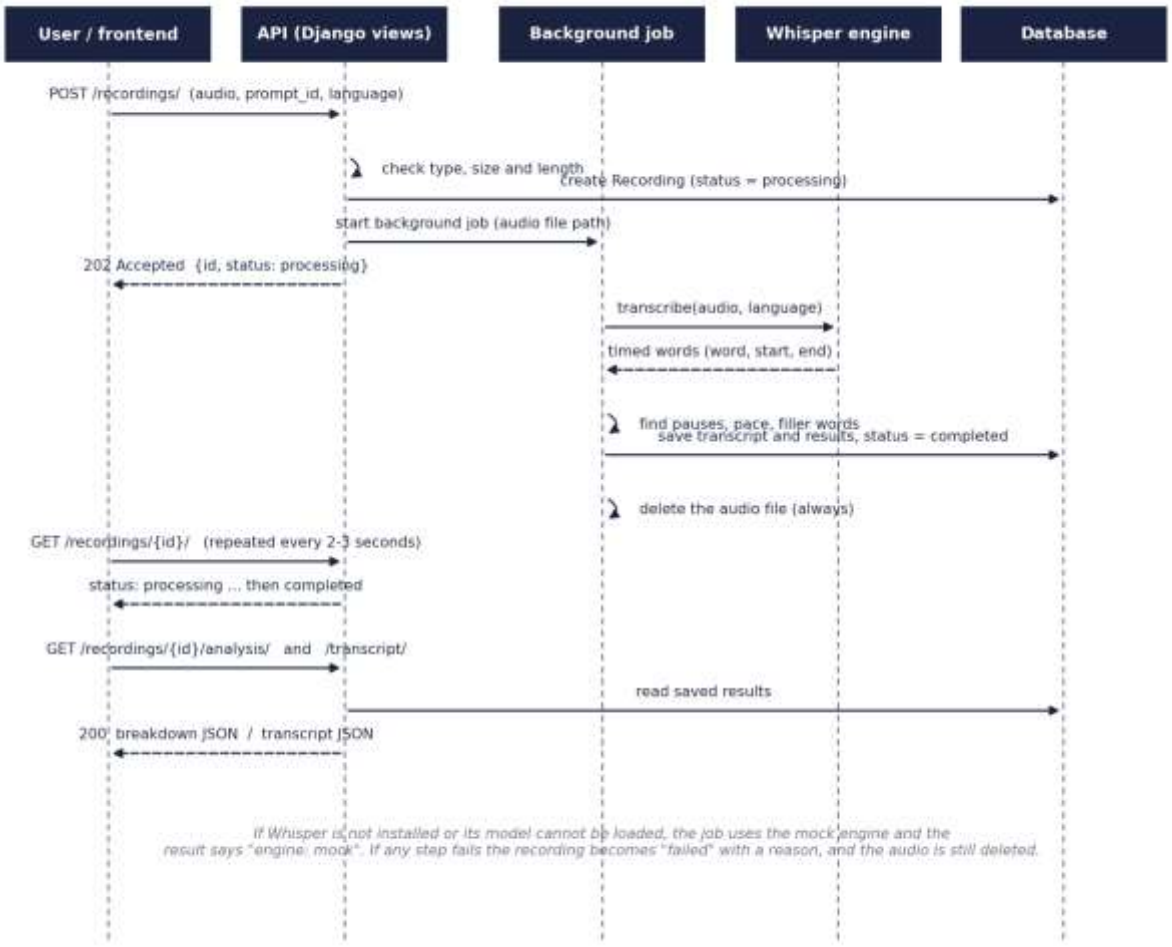


Figure 3: Sequence of one recording

5.1 The rules behind the numbers

- Pace: total words divided by the length in minutes. The pace-over-time chart repeats this for every window of 30 seconds; a very short last window is merged into the one before it.
- Pauses: the gap between the end of one word and the start of the next, using the word times from the speech engine. Gaps under 0.5 s are normal speech. Longer ones are labelled short, dramatic (1 s or more) or long (3 s or more).
- Filler words: words are lowercased and stripped of punctuation, then matched against the active list. Phrases such as "you know" match as consecutive words, longest phrase first. The active list is the global words plus the user's own words for that language.
- Pace status: too slow, within target or too fast, compared with the target range in the settings.
- Failure handling: any error marks the recording failed with a code and a reason. The audio file is deleted in every case.

5.2 Settings an admin can change

Setting	D
max_recording_seconds	30
min_recording_seconds	10
wpm_target_min / wpm_target_max	12
pacing_window_seconds	30
pause_min_seconds	0.
pause_dramatic_min_seconds	1.
pause_long_min_seconds	3.
whisper_model	ba
default_language	en

The numbers for pace and pauses are starting guesses. Each result stores the thresholds it was calculated with, so old results stay explainable after a change.

6. Where the data flows

The context diagram shows the backend as one process with three outside parties. The level 1 diagram opens it up into seven processes and five data stores (Gane and Sarson notation: rounded boxes are processes, open-ended boxes are data stores, plain boxes are outside entities).

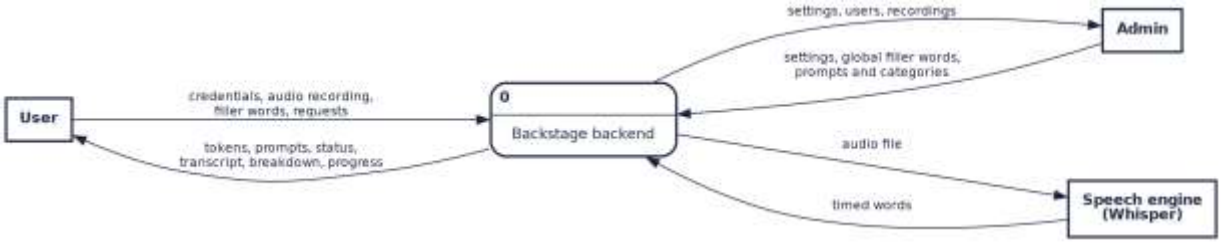


Figure 4: Data flow diagram, level 0 (context)

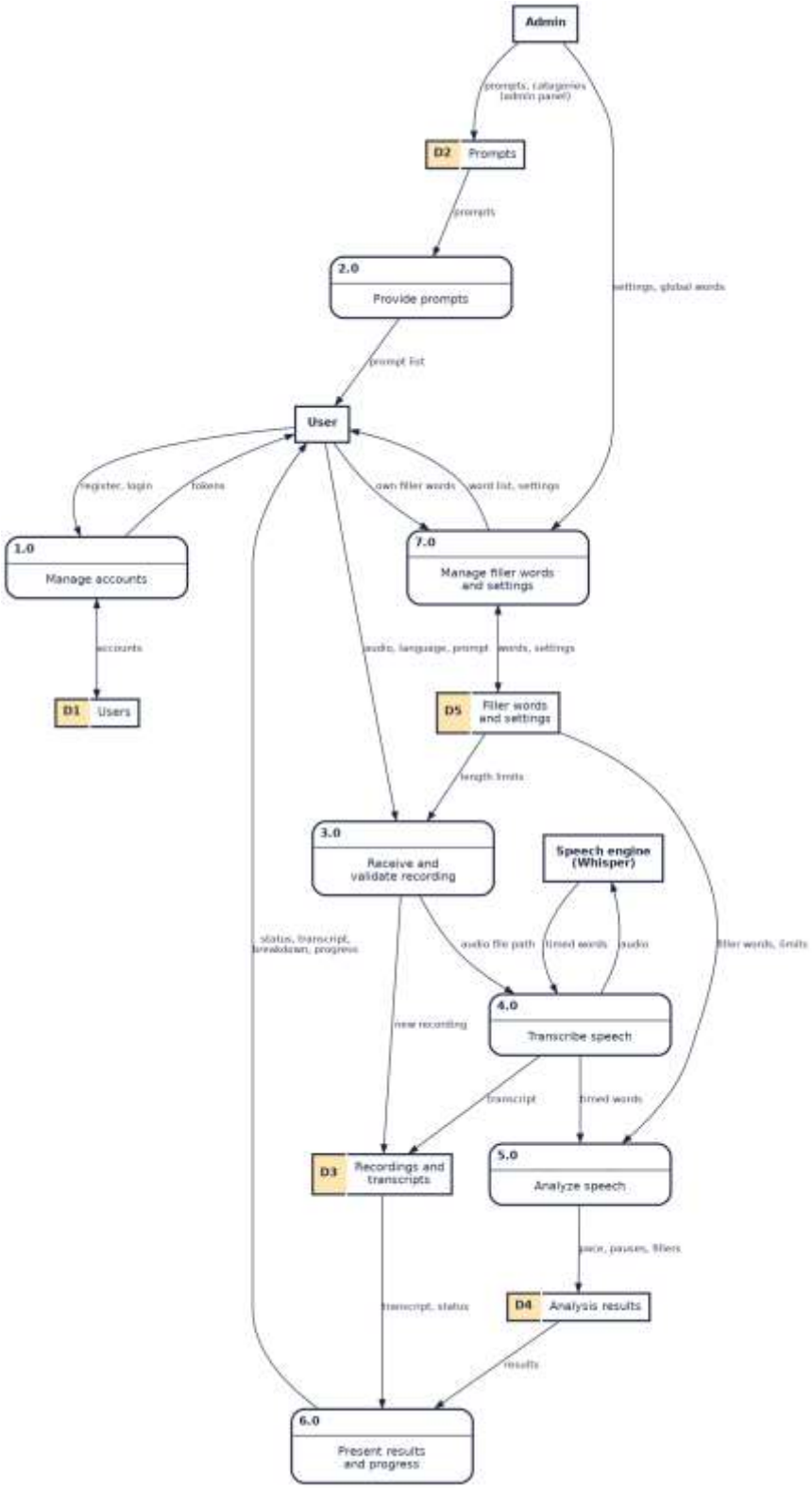


Figure 5: Data flow diagram, level 1

6.1 Processes

No.	P
1.0	M

No.	P
2.0	Pr
3.0	R
4.0	Tr
5.0	A
6.0	Pr
7.0	M

6.2 Data stores

No.	St
D1	U
D2	Pr
D3	R
D4	A
D5	Fi

The audio itself is not in any store. It exists only as a temporary file between processes 3.0 and 5.0 and is removed when processing ends.

7. Data model

Ten tables. Gold tables are the ones admins configure, green tables hold the result of one recording. UK* means the combination of word, language and owner is unique.

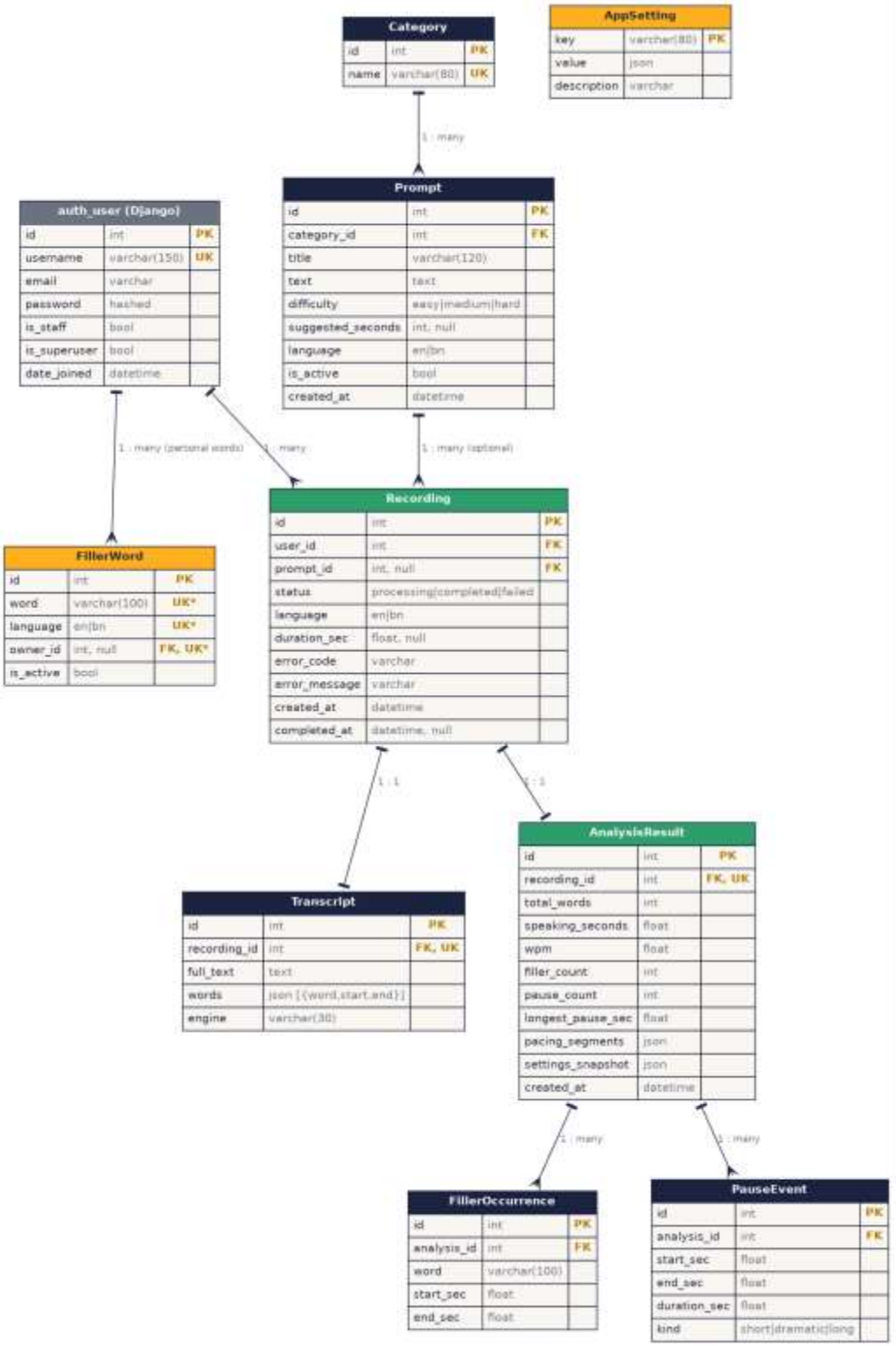


Figure 6: Entity relationship diagram

7.1 Data dictionary

auth_user (Django's own user table)

Field	T
id	in
username	te
email	te
password	te
is_staff / is_superuser	bo
date_joined	da

Category

Field	T
id	in
name	te

Prompt

Field	T
id	in
category_id	FI
title, text	te
difficulty	ea
suggested_seconds	in
language	en
is_active	bo
created_at	da

FillerWord

Field	T
id	in
word	te
language	en
owner_id	FI
is_active	bo

AppSetting

Field	T
key	te
value	js
description	te

Recording

Field	T
id	in
user_id	FK
prompt_id	FK
status	pr
language	en
duration_sec	flo
error_code / error_message	te
created_at / completed_at	da

Transcript

Field	T
id	in
recording_id	FK
full_text	te
words	js
engine	te

AnalysisResult

Field	T
id	in
recording_id	FK
total_words, filler_count, pause_count	in
wpm, speaking_seconds, longest_pause_sec	flo
pacing_segments	js
settings_snapshot	js
created_at	da

FillerOccurrence

Field	T
id	in
analysis_id	FK
word	te
start_sec / end_sec	flo

PauseEvent

Field
id

Field
analysis_id
start_sec / end_sec / duration_sec
kind

8. Interfaces, security and privacy

8.1 API summary

Everything lives under /api/v1/ and speaks JSON, except the audio upload which is multipart form data. After login, every request carries the header Authorization: Bearer <access token>. Lists are paged with page and page_size.

Method	Path
POST	/auth/register/ /auth/login/ /auth/refr
GET	/auth/me/
GET	/categories/ /prompts/ /prompts/{id
POST	/recordings/
GET	/recordings/ /recordings/{id}/
GET	/recordings/{id}/transcript/ /analysis
DELETE	/recordings/{id}/
GET	/progress/
GET POST PATCH DELETE	/filler-words/
GET	/settings/
PATCH	/settings/{key}/

Every error has the same shape: {"error": {"code": "...", "message": "..."}}, with a fields part for validation problems.

HTTP
400
401
403
404
409
413
415
429
500

8.2 Security and privacy

- Passwords are hashed by Django and must pass its password rules. Login, register and upload are rate limited.
- A user can only read, change or delete their own recordings and personal filler words. Other people's items answer "not found".
- Uploads are checked for type, size and length. An oversized upload is refused before it is read.
- Audio is deleted right after processing. Only the transcript and the numbers are kept, and deleting a recording removes them too.

- Only the configured frontend addresses may call the API from a browser (CORS). Secrets live in an .env file that is not committed.
- Demo accounts exist only when seed_demo is run with --users. They are meant for testing, not for a public server.

9. Limits, risks and what is not built

9.1 Tested

- 136 automated tests (services, every endpoint, permissions, uploads, failures) pass on Python 3.9, 3.10, 3.11, 3.12, 3.13 and 3.14.
- A script walks through the whole API (about 20 checks). The test page was clicked through in a simulated browser.
- The team ran the project on Windows with Python 3.13 and real Whisper on an audio file; the transcript, pace and pauses came back as expected.

9.2 Not built (from the PRD)

- Pitch variation, open-ended question tracking, punchline timing and audience-type pacing mode.
- Camera-based features (eye contact, gestures).
- Deployment. The system runs locally and has no public link yet. Whisper needs a computer with enough CPU and memory, which free hosting plans often lack.

9.3 Known limits

- Whisper may leave "um" and "uh" out of a transcript, which lowers filler counts. The first real test with such speech is still to be done and written in the project log. The larger models may help.
- Words like "like", "so" and "right" are often meant normally, so counts can be too high. The word lists are editable for this reason.
- Bangla recognition is less accurate than English, and the Bangla filler list is only a start.
- Processing runs in a thread inside the web process. A server restart interrupts jobs; they are marked failed after 20 minutes and the user uploads again. A job queue would be the next step for many users.
- SQLite is enough for a demo. A real deployment should use PostgreSQL.